

SemiGuard AI - Configuration Files

프로젝트 설정 파일

생성일: Fri Aug 7 11:23:15 UTC 2026

파일: package.json

```
{
  "name": "semiguard-ai",
  "version": "1.0.0",
  "type": "module",
  "license": "MIT",
  "scripts": {
    "dev": "NODE_ENV=development tsx watch server/_core/index.ts",
    "build": "vite build && esbuild server/_core/index.ts --platform=node",
    "start": "NODE_ENV=production node dist/index.js",
    "check": "tsc --noEmit",
    "format": "prettier --write .",
    "test": "vitest run",
    "db:push": "drizzle-kit generate && drizzle-kit migrate"
  },
  "dependencies": {
    "@aws-sdk/client-s3": "^3.693.0",
    "@aws-sdk/s3-request-presigner": "^3.693.0",
    "@hookform/resolvers": "^5.2.2",
    "@radix-ui/react-accordion": "^1.2.12",
    "@radix-ui/react-alert-dialog": "^1.1.15",
    "@radix-ui/react-aspect-ratio": "^1.1.7",
    "@radix-ui/react-avatar": "^1.1.10",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-collapsible": "^1.1.12",
    "@radix-ui/react-context-menu": "^2.2.16",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-dropdown-menu": "^2.1.16",
    "@radix-ui/react-hover-card": "^1.1.15",
    "@radix-ui/react-label": "^2.1.7",
    "@radix-ui/react-menubar": "^1.1.16",
    "@radix-ui/react-navigation-menu": "^1.2.14",
    "@radix-ui/react-popover": "^1.1.15",
    "@radix-ui/react-progress": "^1.1.7",
    "@radix-ui/react-radio-group": "^1.3.8",
    "@radix-ui/react-scroll-area": "^1.2.10",
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-separator": "^1.1.7",
    "@radix-ui/react-slider": "^1.3.6",
    "@radix-ui/react-slot": "^1.2.3",
    "@radix-ui/react-switch": "^1.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
```

```
"@radix-ui/react-toggle": "^1.1.10",
"@radix-ui/react-toggle-group": "^1.1.11",
"@radix-ui/react-tooltip": "^1.2.8",
"@tanstack/react-query": "^5.90.2",
"@trpc/client": "^11.6.0",
"@trpc/react-query": "^11.6.0",
"@trpc/server": "^11.6.0",
"axios": "^1.12.0",
"class-variance-authority": "^0.7.1",
"clsx": "^2.1.1",
"cmdk": "^1.1.1",
"cookie": "^1.0.2",
"date-fns": "^4.1.0",
"dotenv": "^17.2.2",
"drizzle-orm": "^0.44.5",
"embla-carousel-react": "^8.6.0",
"express": "^4.21.2",
"framer-motion": "^12.23.22",
"html-to-image": "^1.11.13",
"html2canvas": "^1.4.1",
"input-otp": "^1.4.2",
"jose": "6.1.0",
"jspdf": "^4.2.1",
"lucide-react": "^0.453.0",
"mysql2": "^3.15.0",
"nanoid": "^5.1.5",
"next-themes": "^0.4.6",
"react": "^19.2.1",
"react-day-picker": "^9.11.1",
"react-dom": "^19.2.1",
"react-hook-form": "^7.64.0",
"react-resizable-panels": "^3.0.6",
"recharts": "^2.15.2",
"sonner": "^2.0.7",
"streamdown": "^1.4.0",
"superjson": "^1.13.3",
"tailwind-merge": "^3.3.1",
"tailwindcss-animate": "^1.0.7",
"vaul": "^1.1.2",
"wouter": "^3.3.5",
"zod": "^4.1.12"
},
"devDependencies": {
  "@builder.io/vite-plugin-jsx-loc": "^0.1.1",
```

```
"@tailwindcss/typography": "^0.5.15",
"@tailwindcss/vite": "^4.1.3",
"@types/express": "4.17.21",
"@types/google.maps": "^3.58.1",
"@types/node": "^24.7.0",
"@types/react": "^19.2.1",
"@types/react-dom": "^19.2.1",
"@vitejs/plugin-react": "^5.0.4",
"add": "^2.0.6",
"autoprefixer": "^10.4.20",
"drizzle-kit": "^0.31.4",
"esbuild": "^0.25.0",
"pnpm": "^10.15.1",
"postcss": "^8.4.47",
"prettier": "^3.6.2",
"tailwindcss": "^4.1.14",
"tsx": "^4.19.1",
"tw-animate-css": "^1.4.0",
"typescript": "5.9.3",
"vite": "^7.1.7",
"vite-plugin-manus-runtime": "0.0.59",
"vitest": "^2.1.4"
},
"packageManager": "pnpm@10.4.1+sha512.c753b6c3ad7afa13af388fa6d808035a0",
"pnpm": {
  "patchedDependencies": {
    "wouter@3.7.1": "patches/wouter@3.7.1.patch"
  },
  "overrides": {
    "tailwindcss>nanoid": "3.3.7"
  }
}
}
```

파일: tsconfig.json

```
{
  "include": ["client/src/**/*", "shared/**/*", "server/**/*"],
  "exclude": ["node_modules", "build", "dist", "**/*.test.ts"],
  "compilerOptions": {
    "incremental": true,
    "tsBuildInfoFile": "./node_modules/typescript/tsbuildinfo",
    "noEmit": true,
    "module": "ESNext",
    "strict": true,
    "lib": ["esnext", "dom", "dom.iterable"],
    "jsx": "preserve",
    "esModuleInterop": true,
    "skipLibCheck": true,
    "allowImportingTsExtensions": true,
    "moduleResolution": "bundler",
    "baseUrl": ".",
    "types": ["node", "vite/client"],
    "paths": {
      "@/*": ["/client/src/*"],
      "@shared/*": ["/shared/*"]
    }
  }
}
```

파일: vite.config.ts

```
import { jsxLocPlugin } from "@builder.io/vite-plugin-jsx-loc";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import fs from "node:fs";
import path from "node:path";
import { defineConfig, type Plugin, type ViteDevServer } from "vite";
import { vitePluginManusRuntime } from "vite-plugin-manus-runtime";

// =====
// Manus Debug Collector - Vite Plugin
// Writes browser logs directly to files, trimmed when exceeding size limit
// =====

const PROJECT_ROOT = import.meta.dirname;
const LOG_DIR = path.join(PROJECT_ROOT, ".manus-logs");
const MAX_LOG_SIZE_BYTES = 1 * 1024 * 1024; // 1MB per log file
const TRIM_TARGET_BYTES = Math.floor(MAX_LOG_SIZE_BYTES * 0.6); // Trim to 60%

type LogSource = "browserConsole" | "networkRequests" | "sessionReplay";

function ensureLogDir() {
  if (!fs.existsSync(LOG_DIR)) {
    fs.mkdirSync(LOG_DIR, { recursive: true });
  }
}

function trimLogFile(logPath: string, maxSize: number) {
  try {
    if (!fs.existsSync(logPath) || fs.statSync(logPath).size ≤ maxSize) {
      return;
    }

    const lines = fs.readFileSync(logPath, "utf-8").split("\n");
    const keptLines: string[] = [];
    let keptBytes = 0;

    // Keep newest lines (from end) that fit within 60% of maxSize
    const targetSize = TRIM_TARGET_BYTES;
    for (let i = lines.length - 1; i ≥ 0; i--) {
      const lineBytes = Buffer.byteLength(`${lines[i]}\n`, "utf-8");
      if (keptBytes + lineBytes > targetSize) break;
    }
  }
}
```

```

        keptLines.unshift(lines[i]);
        keptBytes += lineBytes;
    }

    fs.writeFileSync(logPath, keptLines.join("\n"), "utf-8");
} catch {
    /* ignore trim errors */
}
}

function writeToLogFile(source: LogSource, entries: unknown[]) {
    if (entries.length === 0) return;

    ensureLogDir();
    const logPath = path.join(LOG_DIR, `${source}.log`);

    // Format entries with timestamps
    const lines = entries.map((entry) => {
        const ts = new Date().toISOString();
        return `[${ts}] ${JSON.stringify(entry)}`;
    });

    // Append to log file
    fs.appendFileSync(logPath, `${lines.join("\n")}\n`, "utf-8");

    // Trim if exceeds max size
    trimLogFile(logPath, MAX_LOG_SIZE_BYTES);
}

/**
 * Vite plugin to collect browser debug logs
 * - POST /__manus__/logs: Browser sends logs, written directly to files
 * - Files: browserConsole.log, networkRequests.log, sessionReplay.log
 * - Auto-trimmed when exceeding 1MB (keeps newest entries)
 */
function vitePluginManusDebugCollector(): Plugin {
    return {
        name: "manus-debug-collector",

        transformIndexHtml(html) {
            if (process.env.NODE_ENV === "production") {
                return html;
            }
            return {

```

```

    html,
    tags: [
      {
        tag: "script",
        attrs: {
          src: "/__manus__/debug-collector.js",
          defer: true,
        },
        injectTo: "head",
      },
    ],
  },
};
},

configureServer(server: ViteDevServer) {
  // POST /__manus__/logs: Browser sends logs (written directly to file)
  server.middlewares.use("/__manus__/logs", (req, res, next) => {
    if (req.method !== "POST") {
      return next();
    }

    const handlePayload = (payload: any) => {
      // Write logs directly to files
      if (payload.consoleLogs?.length > 0) {
        writeToLogFile("browserConsole", payload.consoleLogs);
      }
      if (payload.networkRequests?.length > 0) {
        writeToLogFile("networkRequests", payload.networkRequests);
      }
      if (payload.sessionEvents?.length > 0) {
        writeToLogFile("sessionReplay", payload.sessionEvents);
      }

      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ success: true }));
    };

    const reqBody = (req as { body?: unknown }).body;
    if (reqBody && typeof reqBody === "object") {
      try {
        handlePayload(reqBody);
      } catch (e) {
        res.writeHead(400, { "Content-Type": "application/json" });
        res.end(JSON.stringify({ success: false, error: String(e) }));
      }
    }
  });
}

```



```

    }
    return;
  }

  let body = "";
  req.on("data", (chunk) => {
    body += chunk.toString();
  });

  req.on("end", () => {
    try {
      const payload = JSON.parse(body);
      handlePayload(payload);
    } catch (e) {
      res.writeHead(400, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ success: false, error: String(e) }));
    }
  });
});
},
};
};
}

```

```
const plugins = [react(), tailwindcss(), jsxLocPlugin(), vitePluginManusR
```

```

export default defineConfig({
  plugins,
  resolve: {
    alias: {
      "@": path.resolve(import.meta.dirname, "client", "src"),
      "@shared": path.resolve(import.meta.dirname, "shared"),
      "@assets": path.resolve(import.meta.dirname, "attached_assets"),
    },
  },
  envDir: path.resolve(import.meta.dirname),
  root: path.resolve(import.meta.dirname, "client"),
  publicDir: path.resolve(import.meta.dirname, "client", "public"),
  build: {
    outDir: path.resolve(import.meta.dirname, "dist/public"),
    emptyOutDir: true,
  },
  server: {
    host: true,
    allowedHosts: [

```

```
    ".manuspre.computer",
    ".manus.computer",
    ".manus-asia.computer",
    ".manuscomputer.ai",
    ".manusvm.computer",
    "localhost",
    "127.0.0.1",
  ],
  fs: {
    strict: true,
    deny: ["**/*.@"],
  },
},
});
```

파일: vitest.config.ts

```
import { defineConfig } from "vitest/config";
import path from "path";

const templateRoot = path.resolve(import.meta.dirname);

export default defineConfig({
  root: templateRoot,
  resolve: {
    alias: {
      "@": path.resolve(templateRoot, "client", "src"),
      "@shared": path.resolve(templateRoot, "shared"),
      "@assets": path.resolve(templateRoot, "attached_assets"),
    },
  },
  test: {
    environment: "node",
    include: ["server/**/*.test.ts", "server/**/*.spec.ts"],
  },
});
```

파일: .prettierrc

```
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": false,
  "printWidth": 80,
  "tabWidth": 2,
  "useTabs": false,
  "bracketSpacing": true,
  "bracketSameLine": false,
  "arrowParens": "avoid",
  "endOfLine": "\n",
  "quoteProps": "as-needed",
  "jsxSingleQuote": false,
  "proseWrap": "preserve"
}
```

파일: shared/const.ts

```
export const COOKIE_NAME = "app_session_id";
export const ONE_YEAR_MS = 1000 * 60 * 60 * 24 * 365;
export const AXIOS_TIMEOUT_MS = 30_000;
export const UNAUTHED_ERR_MSG = 'Please login (10001)';
export const NOT_ADMIN_ERR_MSG = 'You do not have required permission (10002)';

// One-time nonce cookie that binds an OAuth login to the browser that stole it. The `__Host-` prefix forces the cookie host-only (Secure, Path=/, SameSite=None, Domain), so a sibling *.manus.space site cannot plant a matching value in the victim's browser.
export const OAUTH_STATE_COOKIE = "__Host-oauth_state";

// `state` carries the callback redirect URI (used at token exchange) plus a CSRF nonce. Defined here so the client encoder and server decoder never disagree.
export type OAuthState = { redirectUri: string; nonce?: string };

export const encodeOAuthState = (state: OAuthState): string =>
  btoa(JSON.stringify(state));

export const decodeOAuthState = (state: string): OAuthState => {
  let decoded: string;
  try {
    decoded = atob(state);
  } catch {
    // Malformed base64 (e.g. attacker-supplied garbage). Return no nonce so the
    // callback's CSRF guard rejects it with 403 – never throw, since the error
    // runs outside the request handler's try/catch.
    return { redirectUri: "" };
  }
  try {
    const parsed = JSON.parse(decoded);
    if (parsed && typeof parsed.redirectUri === "string") return parsed;
  } catch {
    // Legacy links: `state` was a bare base64(redirectUri) with no nonce
  }
  return { redirectUri: decoded };
};
```

파일: shared/semiguard.ts

```
// SemiGuard AI 공유 타입 및 상수

export type RiskLevel = "normal" | "caution" | "warning" | "danger";

export interface SensorData {
  current: number;      // 전류 (A)
  temperature: number;  // 온도 (°C)
  vibration: number;     // 진동 (mm/s)
  noise: number;        // 소음 (dB)
  timestamp: number;
}

export interface AnomalyResult {
  sensorData: SensorData;
  anomalyScore: number;  // 0~100
  riskLevel: RiskLevel;
  isAnomaly: boolean;
}

export interface AnomalyLogEntry {
  id: number;
  timestamp: string;
  current: number;
  temperature: number;
  vibration: number;
  noise: number;
  anomalyScore: number;
  riskLevel: RiskLevel;
  isAnomaly: boolean;
  llmAnalysisKo?: string | null;
  llmAnalysisEn?: string | null;
  llmAnalysisJa?: string | null;
}

export const RISK_THRESHOLDS = {
  normal: 29,
  caution: 49,
  warning: 69,
  danger: 100,
} as const;
```

```
export function getRiskLevel(  
  score: number,  
  thresholds: { normal: number; caution: number; warning: number } = RISK,  
): RiskLevel {  
  if (score ≤ thresholds.normal) return "normal";  
  if (score ≤ thresholds.caution) return "caution";  
  if (score ≤ thresholds.warning) return "warning";  
  return "danger";  
}
```